

Phase 3

Digital Audio Loopback

Microphone → STM32 ADC → RAM → MCP4725 DAC → LM386 →
Headphones

Goal: Build a real-time digital audio path through the STM32. The user speaks into the microphone and hears their own voice in the headphones, with every audio sample passing through the microcontroller as a digital number in RAM.

Why this is the most important phase: once it works, the entire local audio pipeline is verified. Every later phase (encryption, LTE transmission, decryption) just changes what happens to the digital samples between ADC and DAC — the hardware path stays the same.

Prerequisites: Phase 1 (analog chain) and Phase 2 (STM32 ADC + LED) must be complete and verified working.

1. Phase 3 Overview

1.1 The Pipeline

Phase 2 (just finished):

mic ---> MAX9814 ---> STM32 ADC ---> [LED on/off] (no audio output)

Phase 3 (this phase):

```

mic ---> MAX9814 ---> STM32 ADC ---> RAM buffer ---> MCP4725 DAC
                                                    |
                                                    v
                                                LM386 amplifier
                                                    |
                                                    v
                                                TRRS jack
                                                    |
                                                    v
                                                Headphones
                                                    |
                                                    v
                                                YOUR EARS

```

1.2 What's New

Item	Purpose
MCP4725 DAC module	Converts digital audio samples back into an analog voltage
I2C1 peripheral	STM32 communicates with the MCP4725 over a 2-wire bus (SDA + SCL)
TIM3 timer	Generates an 8 kHz interrupt — every interrupt writes one sample to the DAC
~15 lines of firmware	TIM3 ISR plus a small change to the main loop

1.3 What Stays the Same

All the wiring from Phases 1 and 2 stays in place:

- MAX9814 powered (3.3 V, GND) and connected to PA0 — unchanged from Phase 2.
- Headset (input) plugged into TRRS#1, with mic wired to MAX9814's empty pads.
- LM386 powered (5 V, GND) and wired to TRRS#2 (output). Same as Phase 1.
- Output headphones still in TRRS#2.
- Black Pill USB-C powering the breadboard rails.

The only signal-path change: the LM386's INPUT switches from *(formerly the MAX9814 OUT)* to *the MCP4725 OUT*. The MAX9814's output now goes to the STM32 instead of straight to the LM386.

2. About the MCP4725 DAC

2.1 What It Does

The MCP4725 is a tiny single-chip DAC (digital-to-analog converter) that turns 12-bit digital values (0–4095) into an analog voltage between 0 V and VDD. It speaks over the I²C bus, so only two wires are needed for control plus VDD and GND for power.

Spec	Value
Resolution	12 bits (4096 voltage levels between 0 V and VDD)
Output range	0 V to VDD (we use VDD = 3.3 V)
Communication	I ² C, 7-bit address 0x60 (when A0 pin is tied to GND)
Maximum I ² C speed	3.4 MHz (we will use 400 kHz Fast Mode — plenty of margin)
Output settling time	~6 μ s — fast enough for 8 kHz audio (period = 125 μ s)
Onboard pull-ups	Most breakouts already include 4.7 k Ω pull-ups on SDA and SCL

2.2 Address Selection

The MCP4725 has an A0 pin that selects between two possible I²C addresses. On most breakouts A0 is already tied to GND on the PCB, giving address **0x60**. If your breakout exposes A0 as a pin/pad, tie it to GND for address 0x60. (If tied to VDD, address would be 0x61 — we will use 0x60 in firmware.)

2.3 Fast-Write Frame Format

To save bandwidth we use the MCP4725's Fast Write mode — just 2 bytes per sample after the address byte:

```
Byte 1:  C2 C1 PD1 PD0 D11 D10 D9 D8      (control bits + upper 4 data bits)
Byte 2:  D7 D6 D5 D4 D3 D2 D1 D0          (lower 8 data bits)
```

For normal operation we send C2=0, C1=0, PD1=0, PD0=0, followed by the 12-bit data value.

At 400 kHz I²C, each 2-byte transfer takes ~25 μ s.
With audio period = 125 μ s per sample at 8 kHz, we have plenty of headroom.

2.4 Why a 12-bit DAC for 8-bit Audio?

The ADC samples are 8-bit (0–255), but the DAC accepts 12-bit values (0–4095). The firmware simply left-shifts the 8-bit value by 4 bits to scale it into the 12-bit range. This wastes some DAC resolution, but it works perfectly for voice quality and keeps the data path simple.

```
/* Convert 8-bit ADC sample to 12-bit DAC value */
uint16_t dac_value = ((uint16_t)adc_sample) << 4;
/* 0    -> 0
   128 -> 2048 (mid-rail)
   255 -> 4080 (close to full scale) */
```

3. Circuit & Wiring

3.1 New Connections (4 wires to the MCP4725)

#	From	To	Notes
1	MCP4725 VDD	3.3 V rail (top + rail)	Powers the DAC chip
2	MCP4725 GND	– rail	Common ground with everything
3	MCP4725 SCL	STM32 PB6 (I2C1_SCL)	I ² C clock
4	MCP4725 SDA	STM32 PB7 (I2C1_SDA)	I ² C data
5	MCP4725 A0	– rail (GND)	Sets I ² C address to 0x60
6	MCP4725 OUT	LM386 IN (header pin)	Replaces the wire that previously came from MAX9814 OUT

3.2 Wire That Must Be Removed

In Phase 2 the MAX9814 OUT was already feeding the STM32 at PA0. In Phase 3 the **LM386 input source changes** from MAX9814 OUT to MCP4725 OUT:

Action	Wire	Reason
REMOVE	Any direct wire from MAX9814 OUT to LM386 IN	Audio now goes through the STM32 first; LM386 input comes from the DAC

3.3 What Stays Wired (do not touch)

- All 4 power rail wires (Black Pill 3V3, 5V, GND).
- MAX9814 power: VDD to 3.3 V rail, GND to – rail.
- TRRS#1 Sleeve to MAX9814 MIC+ pad (the empty pad).
- TRRS#1 Ring 2 to – rail.
- MAX9814 OUT to STM32 PA0 (added in Phase 2).
- LM386 power: VCC to 5 V rail, GND to – rail.
- LM386 green terminal + to TRRS#2 Tip and Ring 1.
- LM386 green terminal – to – rail.

3.4 Final Wiring Summary — All Sections

Section	#	From	To
Power	1	Black Pill 3V3	Top + rail (3.3 V)
	2	Black Pill 5V	Bottom + rail (5 V)
	3	Black Pill GND	– rail
	4	Top – rail	Bottom – rail (common GND)
MAX9814	5	MAX9814 VDD	Top + rail
	6	MAX9814 GND	– rail
	7	TRRS#1 Sleeve	MAX9814 MIC+ empty pad
	8	TRRS#1 Ring 2	– rail
	9	MAX9814 OUT	STM32 PA0 (ADC1_IN0)

Section	#	From	To
MCP4725	10	MCP4725 VDD	Top + rail (3.3 V)
	11	MCP4725 GND	– rail
	12	MCP4725 SCL	STM32 PB6
	13	MCP4725 SDA	STM32 PB7
	14	MCP4725 A0	– rail (selects address 0x60)
	15	MCP4725 OUT	LM386 IN (header pin)
LM386	16	LM386 VCC	Bottom + rail (5 V)
	17	LM386 GND	– rail
	18	LM386 + (green)	TRRS#2 Tip and Ring 1
	19	LM386 – (green)	– rail

Total: **19 wires**. No extra capacitors, no extra resistors — the MAX9814 OUT now goes directly to PA0 (the STM32 ADC input has its own bias tolerance), and the MCP4725 OUT goes directly to LM386 IN (modern LM386 modules include their own input coupling capacitor).

4. STM32CubeMX Configuration

Open your existing **MicLED** project's .ioc file in CubeMX. Add I2C1 and TIM3, then regenerate code.

4.1 Add I2C1 (for the MCP4725)

- Connectivity → **I2C1** → Mode: **I2C**.
- PB6 turns yellow as SCL, PB7 turns yellow as SDA on the chip diagram.
- Parameter Settings → I2C Speed Mode: **Fast Mode**.
- Parameter Settings → I2C Clock Speed (Hz): **400000**.
- Parameter Settings → Fast Mode Duty Cycle: 2.
- Other defaults are fine.
- (Optional but recommended) DMA Settings → Add → **I2C1_TX**: Mode = Normal, Direction = Memory To Peripheral, Byte width.

4.2 Add TIM3 (8 kHz Playback Tick)

- Timers → **TIM3** → Clock Source: **Internal Clock**.
- Parameter Settings → Prescaler: **83**.
- Parameter Settings → Counter Period: **124**.
- Parameter Settings → Auto-reload preload: **Enable**.
- **NVIC Settings tab** → enable **TIM3 global interrupt**.

Math: TIM3 clock = 84 MHz / ((83+1) × (124+1)) = 8000 Hz exact — same as TIM2 already configured for the ADC.

4.3 NVIC Priority Order

- System Core → NVIC → verify these enabled with the listed priorities:

Interrupt	Preempt Priority	Why
DMA2 Stream0 (ADC)	0 (highest)	Audio capture must never be starved
I2C1 EV	1	DAC writes must complete before next TIM3 tick
I2C1 ER	1	I ² C error handling
ADC1	2	Conversion-complete signaling
TIM3	2	Playback tick

4.4 Generate Code

- Click **Project Manager** tab → verify Toolchain is still **CMake**.
- Click **GENERATE CODE**.
- When done, return to VS Code — the new files (i2c.c, i2c.h, tim.h additions) appear in Core/Src and Core/Inc.

5. Firmware Code

Open **Core/Src/main.c**. Add the snippets below at the marked USER CODE regions. Keep all the Phase 2 code that is already there — we are **adding**, not replacing.

5.1 Add to USER CODE BEGIN PD (private defines)

```
/* USER CODE BEGIN PD */
#define BUF_SIZE      256
#define HALF_SIZE     (BUF_SIZE / 2)
#define VOICE_THRESHOLD 20
#define MCP4725_ADDR  (0x60 << 1) /* HAL expects shifted 8-bit address */
/* USER CODE END PD */
```

5.2 Add to USER CODE BEGIN PV (private variables)

```
/* USER CODE BEGIN PV */
uint8_t  adc_buffer[BUF_SIZE];
volatile uint8_t half_ready = 0;
volatile uint8_t full_ready = 0;
volatile uint16_t dac_play_idx = 0; /* index into adc_buffer for next DAC sample */
/* USER CODE END PV */
```

5.3 Add to USER CODE BEGIN 2 (after MX_*_Init() calls)

```
/* USER CODE BEGIN 2 */
HAL_ADC_Start_DMA(&hadc1, (uint32_t*)adc_buffer, BUF_SIZE);
HAL_TIM_Base_Start(&htim2); /* drives the 8 kHz ADC trigger */
HAL_TIM_Base_Start_IT(&htim3); /* drives the 8 kHz DAC playback */
/* USER CODE END 2 */
```

5.4 Replace the WHILE loop (USER CODE BEGIN WHILE)

In Phase 2 the loop drove the LED. In Phase 3 the loop just toggles the heartbeat LED — the audio playback runs entirely in interrupts.

```
/* USER CODE BEGIN WHILE */
while (1)
{
    /* Optional: heartbeat LED so we know firmware is alive */
    if (half_ready || full_ready) {
        half_ready = 0;
        full_ready = 0;
        HAL_GPIO_TogglePin(GPIOC, GPIO_PIN_13);
    }
}
/* USER CODE END WHILE */
```

5.5 Add the TIM3 ISR — the heart of Phase 3

This callback runs 8000 times per second. Each call grabs the latest ADC sample from RAM and writes it to the MCP4725 over I²C.

```
/* In USER CODE BEGIN 4 (or extend it if Phase 2 already added callbacks) */
void HAL_ADC_ConvHalfCpltCallback(ADC_HandleTypeDef *hadc) {
    if (hadc->Instance == ADC1) half_ready = 1;
}
void HAL_ADC_ConvCpltCallback(ADC_HandleTypeDef *hadc) {
    if (hadc->Instance == ADC1) full_ready = 1;
}

void HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *htim) {
    if (htim->Instance == TIM3) {
        /* Take the next ADC sample (8-bit) and scale it to 12-bit for MCP4725 */
        uint16_t v12 = ((uint16_t)adc_buffer[dac_play_idx]) << 4;
```

```
/* MCP4725 Fast-Write packet (2 bytes) */
uint8_t pkt[2] = {
    (uint8_t)((v12 >> 8) & 0x0F), /* upper 4 bits */
    (uint8_t)(v12 & 0xFF)         /* lower 8 bits */
};
HAL_I2C_Master_Transmit_IT(&hi2c1, MCP4725_ADDR, pkt, 2);

/* Advance the playback index, wrap at end of buffer */
dac_play_idx = (dac_play_idx + 1) % BUF_SIZE;
}
}
```

Why HAL_I2C_Master_Transmit_IT (interrupt) and not the blocking version? At 400 kHz the 2-byte transfer takes ~25 μ s. Blocking inside a TIM3 ISR for that long would block other interrupts. The IT (interrupt) version starts the transfer and returns immediately; the I²C peripheral finishes in the background while the rest of the firmware runs.

6. Build, Flash & Test

6.1 Build

- Press **Ctrl + S** in VS Code to save main.c.
- Press **Ctrl + Shift + B** → pick **Build**.
- Watch the terminal for the build output:

```
[16/16] Linking C executable MicLED.elf
      text      data      bss      dec      hex filename
      ...
Build finished. 0 errors, 0 warnings.
```

6.2 Flash

- Verify wiring: ST-Link SWDIO/SWCLK/GND to Black Pill, board powered via USB-C.
- Verify breadboard: all 19 wires from Section 3.4 in place, MCP4725 properly seated.
- In VS Code STM32 sidebar, click **Run**. Or use Run/Debug → green arrow.
- Terminal shows:

```
Memory Programming ...
File download complete
Time elapsed during download operation: ~1s
Application is running, Reset device.
```

6.3 Test Procedure

- Plug the input headset into TRRS#1 (with mic wired to MAX9814 MIC+ empty pad).
- Plug the output headphones into TRRS#2.
- Set LM386 gain knob to **minimum** (fully counter-clockwise).
- Hold the input headset and output headphones **physically apart** from each other (acoustic feedback prevention).
- Place one earbud loosely near the ear — do not insert.
- Slowly turn the LM386 gain clockwise while speaking softly into the input headset's mic.
- When the voice becomes audible at low gain, the loop is working.

6.4 Expected Results

Stimulus	Expected Result
Silence	Quiet hiss in earbud (background noise of the analog chain)
Speaking normally	Your own voice reproduced in the earbud, with imperceptible delay (~30 ms)
Whistling	Whistle reproduced clearly
Sharp clap	Clap reproduced as a short impulse
Tap the input mic with finger	Each tap reproduced as a "thump"
PC13 onboard LED	Toggles every 16 ms when audio frames arrive (looks like steady glow at low brightness)

Pass criterion: Voice audible in the headphones, with the audio passing through the STM32 (you can confirm this by simply removing power to the Black Pill — the audio cuts out, proving the path goes through the microcontroller).

7. Troubleshooting

Symptom	Likely Cause	Fix
Build error: hi2c1 undeclared	I2C1 not added in CubeMX or code not regenerated	Re-open .ioc, ensure I2C1 enabled, click Generate Code
Build error: htim3 undeclared	TIM3 not added in CubeMX	Re-open .ioc, enable TIM3 internal clock, regenerate
Total silence in headphones	(a) MCP4725 not powered, (b) wrong I ² C address, (c) MCP4725 OUT not connected to LM386 IN	Multimeter: 3.3V on MCP4725 VDD; verify A0 to GND; verify wire 15 from Section 3.4
Distorted / harsh / clipping sound	LM386 gain too high; or DAC saturating because input audio is louder than 8-bit range	Lower LM386 gain; speak softer; if still bad, divide ADC sample by 2 in firmware
Steady tone instead of voice	TIM3 firing too fast or I ² C transactions colliding	Verify TIM3 PSC=83, ARR=124; verify I2C1 at 400 kHz Fast Mode
Audio crackles / drops out	I ² C not finishing in time (busy too long)	Verify I2C1 at 400 kHz; verify NVIC priority of TIM3 lower than I2C1
Audio plays but sounds robotic / aliased	ADC offset (~96/255) being treated as audio	Subtract 96 from sample before scaling: $v12 = ((adc_buffer[i] - 96 + 128) \& 0xFF) \ll 4$
HAL_I2C_Master_Transmit_IT returns BUSY	Previous transfer still in progress	Check that NVIC priorities are correct (I2C IRQ above TIM3); add a small skip-if-busy guard
LED on PC13 stays off forever	Heartbeat code not running — main loop blocked	ADC DMA may be misconfigured (Circular mode? NVIC enabled?)

7.1 Quick Diagnostic Sequence

- **Power check:** multimeter on MCP4725 VDD—GND should read 3.3 V.
- **I²C check:** add a simple I2C device scan loop in firmware that walks addresses 1–127 and prints anything found over USART2 / debug. Should report 0x60.
- **DC test:** temporarily replace the TIM3 ISR body with two alternating writes (one to 0x000, one to 0xFFFF, with a 1-second HAL_Delay between). Multimeter on MCP4725 OUT should toggle between ~0 V and ~3.3 V.
- **Tone test:** generate a 500 Hz square wave on the DAC. You should hear a clean low-pitched beep.
- **Loopback test:** only after all of the above pass, run the full firmware and expect to hear yourself.

7.2 What This Phase Proves

- I²C peripheral is correctly configured at 400 kHz.
- The MCP4725 DAC is responding at address 0x60.
- TIM3 fires at exactly 8 kHz, locked to the same clock as TIM2 (the ADC trigger).
- Interrupt-driven I²C transmission completes within the 125 μs budget every cycle.
- ADC and DAC sample rates are synchronised — samples are produced and consumed at the same rate, so the buffer never overruns or underruns.
- The full real-time digital audio loop closes: every sample passes through the STM32's RAM.
- The system has 4 ms of buffer latency (256 samples / 8 kHz / 2 halves) plus the ADC + I²C processing time.

These are exactly the primitives every later phase will build on. Phase 4 (LTE manual AT commands) and beyond add a network in the middle of the loop, but the audio path itself stays identical.

8. After Phase 3

Once you can hear yourself reliably through the digital loopback, the upcoming phases are:

Phase	Description	New Hardware	New Firmware Concept
Phase 4	LTE module bring-up — manual AT commands via USB-TTL	T-A7608 LTE module + USB-TTL adapter	None — you type AT commands by hand to verify the modem
Phase 5	STM32 drives the LTE module	Wire LTE module to STM32 USART1	AT-command state machine + relay TCP connection
Phase 6	Two-node plain audio over LTE	Both devices fully assembled	Send the encrypted-audio bytes over the TCP channel (no encryption yet)
Phase 7	Add hybrid encryption	No new hardware	mbedTLS RSA + AES-128-CTR; key exchange at session start
Phase 8	Polish — phone-number-based call setup, status LEDs, push buttons	2 push buttons per device	Call/Answer/Hangup state machine, blinking-LED state indicator

End of Phase 3 document.